Heterogeneous Tensor Parallelism over Commodity TCP Networks via FP32 GPU-Native Reduction

Brian Habana
BareMetalRT
brian@baremetalrt.com

Abstract

We present the first system that achieves **heterogeneous tensor parallelism on ad-hoc consumer GPUs over commodity TCP networks**, running Mistral 7B—a 14 GB model too large for either GPU alone—across an RTX 4070 Super (12 GB) and an RTX 4060 Laptop (8 GB) at **80 ms/token (12.5 tok/s)**. Every prior distributed inference project on consumer hardware chose pipeline parallelism, accepting idle GPUs and architectural limitations as unavoidable. Data-center systems (HexGen, Helix) support heterogeneous tensor parallelism but require managed clusters, Linux, and offline profiling.

The central contribution is a precision result: FP32 end-to-end computation achieves the **theoretical floor** of IEEE 754 single-precision arithmetic—identical to NCCL on NVLink and $2{,}500\times$ more accurate than FP16. No implementation of tensor parallelism at FP32 precision, on any hardware, can do better. An asymmetric-tolerant TCP transport replaces NCCL, enabling GPUs with different VRAM and compute capabilities to participate in the same AllReduce without barrier stalls or protocol-level assumptions of symmetric hardware. The system is built on NVIDIA TensorRT-LLM ported natively to Windows.

Our benchmarks reveal a counterintuitive finding: in synchronous tensor parallelism, **GPU synchronization overhead—not network latency—is the dominant bottleneck**. A $300\times$ improvement in network speed (WiFi to ethernet) yielded zero throughput improvement. We argue that Mixture-of-Experts architectures structurally favor distributed meshes over centralized data centers, replacing dense AllReduce with sparse expert routing over commodity networks.

1 Introduction

A 7-billion-parameter model doesn't fit on most consumer GPUs. A 70-billion-parameter model doesn't fit on *any* of them. But most households with a gaming PC have more than enough aggregate VRAM across machines to run these models—if those GPUs could work together. They can't. Over 100 million RTX-class NVIDIA GPUs sit in PCs and workstations [NVIDIA, 2024c], and almost none of them can combine their memory and compute with the GPU in the next room, let alone across a network.

This isn't a missing feature—it's a missing *category*. Every existing path forces a choice between quality and accessibility. Cloud inference (OpenAI, Anthropic, Google) uses data-center-grade hardware with optimized kernels—but is expensive, rate-limited, and geographically constrained. Local inference tools (Ollama, LM Studio, Jan, GPT4All) run on consumer hardware—but are limited to a single machine, use a general-purpose CUDA backend that leaves significant performance on the table, and can only run models that fit in one GPU's memory. Distributed inference projects (Petals [Borzunov et al., 2023], Exo [Exo Explore, 2024]) can span multiple machines—but only via pipeline parallelism, which wastes half the available compute to pipeline bubbles and limits model architectures. The fastest inference kernels—NVIDIA's TensorRT-LLM [NVIDIA, 2024d], with 1,500+ hand-optimized CUDA routines—exist, but require homogeneous data-center GPUs communicating over NVLink or InfiniBand via NCCL [NVIDIA, 2024b].

No system combines the three things needed at once: **optimized kernels** that extract full performance from the hardware, **tensor parallelism** that keeps every GPU busy on every layer, and **heterogeneous support** that lets different GPUs with different VRAM and different compute capabilities—on different machines, over a commodity network—contribute to a single inference pass. That is the problem this paper solves. Moreover, the industry's shift toward Mixture-of-Experts (MoE) architectures—which activate only a fraction of parameters per token—creates a structural advantage for distributed meshes over centralized data centers, a direction we explore in §9.

2 Related Work

Parallelism strategies. Model parallelism for large neural networks has been explored along two axes. Pipeline parallelism (PP), introduced by GPipe [Huang et al., 2019], partitions a model into sequential stages assigned to different devices. Megatron-LM [Shoeybi et al., 2019] introduced intra-layer tensor parallelism (TP) for transformer models, splitting attention and MLP layers horizontally across GPUs connected by NVLink within a single DGX node. DeepSpeed [Rasley et al., 2020] combines PP with ZeRO-style memory optimization for training at scale. Alpa [Zheng et al., 2022] automates the selection of inter- and intra-operator parallelism strategies via ILP. All of these systems assume homogeneous hardware within each parallelism group and rely on NCCL [NVIDIA, 2024b] for collective communication—requiring Linux, NVLink or InfiniBand, and identical GPUs.

Distributed inference on consumer hardware. Petals [Borzunov et al., 2023] enables collaborative inference by distributing transformer blocks across volunteers over the internet, using pipeline parallelism exclusively. The authors explicitly state that tensor parallelism is infeasible in their setting due to per-layer activation transfers. Exo [Exo Explore, 2024] similarly builds on pipeline parallelism for home clusters, acknowledging that inter-device latency makes per-layer synchronization impractical. Both systems accept pipeline bubbles and sequential stage execution as necessary tradeoffs. Neither supports heterogeneous GPUs within a parallelism group, and neither addresses the precision loss that occurs when AllReduce is performed over a network rather than NVLink.

Offloading and single-GPU approaches. FlexGen [Sheng et al., 2023] enables large model inference on a single GPU by offloading weights to CPU and disk, trading latency for accessibility. `llama.cpp` provides CPU and CUDA inference via a general-purpose backend but does not use TensorRT-LLM's optimized kernels and does not support multi-machine parallelism. These approaches are complementary: they maximize single-device utilization, while our work enables models too large for any single device.

Heterogeneous parallelism. AMP [Lee et al., 2022] explores asymmetric partitioning for training on heterogeneous GPU clusters, assigning different numbers of layers to devices based on compute capability—a form of asymmetric pipeline parallelism, not tensor parallelism. HexGen [Jiang et al., 2024b] supports asymmetric tensor and pipeline parallelism over heterogeneous GPUs and networks, using constrained optimization to assign workloads proportionally to device capability. Helix [Mei et al., 2025] formulates heterogeneous LLM serving as a max-flow problem over weighted GPU graphs. Both achieve significant throughput improvements over homogeneous baselines—but both assume managed datacenter clusters with known topology, require offline profiling and optimization passes, and build on existing Linux-only frameworks (vLLM, Megatron). Neither targets ad-hoc consumer hardware, WiFi or commodity TCP networks, or Windows. Our work demonstrates heterogeneous tensor parallelism in the setting these systems explicitly do not address: consumer GPUs with different VRAM and SM counts, discovered dynamically, communicating over commodity TCP without NVLink or InfiniBand—and does so with FP32-accurate AllReduce that prevents the precision loss inherent in networked reduction.

3 Approach: FP32 Reduction and Asymmetric Tensor Parallelism

Every prior attempt at distributed LLM inference over consumer hardware chose pipeline parallelism—and accepted the bubbles, idle GPUs, and architectural limitations that come with it. Tensor parallelism was considered infeasible outside of homogeneous data center hardware. Our system solves the two problems that blocked it: **precision loss during networked AllReduce**, and **synchronization failure across mismatched GPUs**. Together, they enable heterogeneous tensor parallelism on consumer hardware over commodity networks—a setting no prior system has addressed.

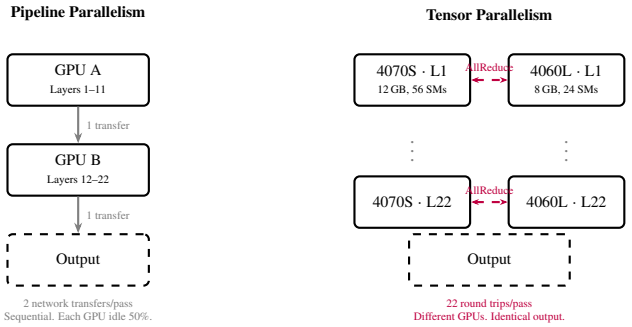

Figure 1: Pipeline parallelism (left) assigns contiguous layer blocks to each GPU with 2 network transfers per pass but 50% GPU idle time. Tensor parallelism (right) splits every layer across both GPUs, requiring an AllReduce at each layer boundary—22 round trips for a 22-layer model—but keeps all GPUs active on every layer.

As described in §2, pipeline parallelism (PP) [Huang et al., 2019] splits models vertically across stages, while tensor parallelism (TP) [Shoeybi et al., 2019] splits each layer horizontally, keeping every GPU active on every layer. TP requires AllReduce synchronization at every layer boundary—and every existing implementation assumes identical GPUs.

The consensus in prior work is that per-layer synchronization makes TP infeasible over commodity networks (§2). Over WiFi (1–5 ms per hop), a 32-layer model pays 32 round trips per token; over NVLink (600 ns), that cost is invisible. This assumption drove every previous project to choose PP. As our benchmarks reveal (§7.4), **GPU synchronization overhead, not network latency, dominates the per-layer cost**—but latency was only part of the problem. Behind it was a second, harder wall: **precision loss during AllReduce**. We address precision in its own section (§4) because it is the central technical contribution of this work.

The second innovation is what makes heterogeneous TP possible: a **transport layer that tolerates asymmetric compute**. NCCL's AllReduce algorithms—ring, tree, recursive halving—assume all ranks complete each layer at roughly the same time. When a 4070 Super finishes a layer in 2 ms and a 4060 Laptop takes 4 ms, NCCL's synchronization model breaks down. Our TCP transport doesn't assume symmetric timing. Each rank computes at its own speed, signals readiness, and the exchange happens when both sides are done. The slower GPU sets the compute pace, and network latency adds overhead at every synchronization point—but the faster GPU doesn't stall the protocol, it simply waits on a non-blocking receive. This is what enables two GPUs with different SM counts, different VRAM, and different clock speeds to produce identical results.

Heterogeneous GPUs differ in SM count, clock speed, memory bandwidth, and tensor core count—all of which affect

per-layer compute time. In a standard ring AllReduce [Gibiansky, 2017], if one rank finishes its chunk while another is still computing, the ring stalls. NCCL's collective algorithms were not designed for asymmetric participants. The cost of our approach is real: the slowest GPU sets the pace. But a 4070 Super (12 GB) paired with a 4060 Laptop (8 GB) can run a model too large for either card alone—the tradeoff is **reduced throughput versus inability to run the model at all**.

The technique is also transport-agnostic. We demonstrate it over TCP, but the same asymmetric-tolerant protocol could enable heterogeneous TP over NVLink or PCIe—a configuration NCCL does not support today, even within a data center.

4 Precision Correctness: The Theoretical Floor

Precision correctness is the central technical challenge of tensor parallelism over commodity hardware—and the reason every prior consumer-GPU project chose pipeline parallelism instead. This section presents the problem, the IEEE 754 arithmetic that governs it, our measurements across two models, and the design that achieves the theoretical floor.

4.1 Why Tensor Parallelism Requires Precision That Pipeline Parallelism Does Not

In pipeline parallelism, each GPU runs a contiguous block of layers end-to-end and passes the final hidden state to the next GPU. The hand-off is a simple copy—no arithmetic is performed on the activation during transfer. The only precision requirement is that the tensor is faithfully serialized and deserialized, which any data type satisfies trivially.

Tensor parallelism is fundamentally different. Each layer's computation is split across GPUs, and an AllReduce operation—a summation of partial results from every rank—must execute at every layer boundary. A typical LLM has 22–80 transformer layers, each containing two AllReduce points: one after the attention projection and one after the MLP. A 32-layer model performs **64 AllReduce operations per token**. Every one of these operations is an arithmetic reduction—and every reduction is subject to floating-point rounding. If the rounding error is large enough, it compounds through subsequent layers until the model produces wrong output.

4.2 IEEE 754: 10 Bits vs 23 Bits

The difference between FP16 and FP32 is not a matter of degree. It is structural.

IEEE 754 half-precision (FP16) stores 10 bits of mantissa, giving ∼3.3 decimal digits of precision. IEEE 754 single-precision (FP32) stores 23 bits of mantissa, giving ∼7.2 decimal digits. When two floating-point numbers are added, the smaller value is right-shifted to align exponents, and the trailing bits that fall off the mantissa are lost. In FP16, any addition where the operands differ by more than a factor of ∼1,024 (2^{10}) discards the smaller value entirely. In FP32, this threshold is ∼8.4 million (2^{23}).

Table 1: TinyLlama 1.1B—FP32 vs FP16 model (22 layers, TP=2 simulated). Prompt: "The capital of France is". Error = mean absolute difference of activation tensors vs. FP32 single-GPU baseline. Both produce "Paris" for this short prompt.

Layer	FP32 Error	FP16 Error	Ratio
Layer 1	1.64×10^{-9}	4.91×10^{-6}	2,996×
Layer 5	2.64×10^{-8}	6.08×10^{-5}	2,301×
Layer 10	4.60×10^{-8}	1.22×10^{-4}	2,661×
Layer 15	7.02×10^{-8}	1.72×10^{-4}	2,447×
Layer 22	$\mathbf{2.62\times10^{-7}}$	$\mathbf{6.73\times10^{-4}}$	**2,572×**

Table 2: Mistral 7B—FP16 model (32 layers, TP=2 simulated), same prompt. Error grows from 1.66×10^{-6} at layer 1 to 5.23×10^{-4} at layer 32—a 315× increase across the network. Activation magnitudes reach 252.

Layer	TP=2 Error vs Baseline	Max |activation|
Layer 1	1.66×10^{-6}	0.2
Layer 10	3.86×10^{-5}	247.4
Layer 20	1.32×10^{-4}	251.9
Layer 32	$\mathbf{5.23\times10^{-4}}$	**181.0**

In a row-parallel linear layer, each rank computes a partial matrix product and AllReduce sums the partials. These partial sums can differ by orders of magnitude—some ranks hold weight slices with large values, others with small values. FP16 summation silently drops the contribution of the smaller partial. FP32 preserves it.

This is not a theoretical concern. It is measurable at every layer of a real model.

4.3 Measurements: Activation Drift Across Layers

We simulate TP=2 on a single GPU by splitting each row-parallel linear layer (attention output projection and MLP down projection) into two halves along the input dimension, computing each half independently, and summing via AllReduce. We compare the resulting hidden states at every layer against a single-GPU baseline. The experiment isolates the effect of the TP split and reduction—all other computation is identical.

Table 1 is the key result. The FP32 column represents the **irreducible error** introduced by splitting a matrix multiply across two ranks. At layer 22, the mean absolute error is 2.62×10^{-7}—seven orders of magnitude smaller than the activation values themselves. This is not a quality gap that could be improved with engineering. It is the **theoretical floor** of IEEE 754 single-precision arithmetic.

The floor exists because floating-point addition is not associative. A matrix multiply $Y = X \cdot W$ computed as a single operation produces $Y = \sum x_i w_i$ with one specific summation order. When the same multiply is split across

two ranks—rank 0 computing $\sum_{i<N/2}$ and rank 1 computing $\sum_{i\geq N/2}$—the partial sums are added in a different order. The difference is the rounding noise inherent in FP32's 23-bit mantissa. **Every distributed system has this floor, including NCCL on NVLink.** There is no implementation of tensor parallelism, on any hardware, that avoids it.

FP64 (double-precision) would reduce the floor to $\sim 10^{-16}$ at twice the memory cost and roughly half the compute throughput. The improvement is real but irrelevant: 10^{-7} error cannot flip a logit. The gap between the top-1 and top-2 token in a confident prediction is typically 1–10 in logit space. An error of 10^{-7} is ten million times smaller than the decision boundary. **FP32 is effectively lossless for tensor-parallel AllReduce.**

4.4 The FP16 Failure Mode: Drift and Overflow

FP16 is not close to the floor. It is $2{,}500\times$ above it.

At every layer, FP16 computation introduces $\sim 2{,}500\times$ more activation error than FP32 (Table 1, ratio column). The error is not in the AllReduce alone—on GPU, CUDA promotes FP16 additions to FP32 internally, so two-operand FP16 reduction is lossless. The error accumulates in the *model computation* between AllReduce calls: the matrix multiplies, the residual additions, the layer normalizations. Each of these operations loses bits when performed in FP16, and the losses compound through every subsequent layer.

On TinyLlama (22 layers), the accumulated FP16 error reaches 6.73×10^{-4} at the final layer. For the prompt "The capital of France is", this is not enough to flip the output—"Paris" dominates the logit distribution by a wide margin. But on harder prompts where the top-2 tokens are separated by a smaller gap—reasoning chains, code generation, ambiguous completions—an error of this magnitude can and will flip the winner. And once one wrong token enters the autoregressive loop, every subsequent token is conditioned on a mistake. The error doesn't correct itself. It cascades.

On Mistral 7B (32 layers, Table 2), the error grows to 5.23×10^{-4} with activation magnitudes reaching 252. This is still within FP16's representable range (max 65,504)—but under adversarial conditions, the partial sums at AllReduce boundaries can be much larger. In our early TP experiments, CPU-side FP16 reduction—summing two partial activation tensors in half-precision on the host—produced **outright NaN at layer 23**. IEEE 754 half-precision overflows at 65,504; two large partial sums exceeded this range, producing Inf, which propagated as NaN through every remaining layer. The failure was silent: no error message, no warning, just garbage output. This is the class of failure that would face any system attempting FP16 tensor parallelism over a network—and the reason no prior project attempted it. Petals [Borzunov et al., 2023] and Exo [Exo Explore, 2024] chose pipeline parallelism precisely to avoid per-layer reduction, sidestepping the problem rather than solving it.

4.5 Design: FP32 End-to-End

We solve this with two design choices that work together. First, **the entire computation path runs in FP32**. Weights are stored in FP16 for memory efficiency, but every matmul, every residual connection, and every normalization executes in FP32 precision. This is the key insight: FP32 AllReduce alone is not sufficient. If the model computes in FP16 and only the reduction is FP32, the drift accumulates in the model layers between reductions. The entire computation path must be FP32 to reach the theoretical floor.

A custom CUDA kernel performs the AllReduce reduction on the GPU in FP32; the math never touches the CPU, eliminating the overflow path entirely. Even when the AllReduce operands arrive in FP16 (from a future mixed-precision mode), the kernel promotes to FP32 before accumulating and converts back after—matching NCCL's internal behavior.

Second, explicit `.cast(residual.dtype)` operations at each residual connection ensure that AllReduce outputs are type-compatible with the residual stream. Without these casts, TRT-LLM's graph optimizer silently inserts FP16 conversions that reintroduce precision loss at the exact points where it matters most. Three lines of code in the model definition—but without them, the output is wrong.

4.6 Live Validation

We validate the precision design on a real cross-machine TP=2 deployment: an RTX 4070 Super (desktop, 12 GB, 56 SMs) and an RTX 4060 Laptop (8 GB, 24 SMs) running Mistral 7B Instruct (14 GB total—too large for either GPU alone) over ethernet. Three test prompts:

- "1+1=" → **"2"**
- "The capital of France is" → **"Paris."**
- "A cat is a" → **"small domesticated mammal known for its playful, curious and independent nature…"** (120 tokens, coherent)

All three produce correct, coherent output at $\sim$80 ms/token. The third prompt is particularly meaningful: 120 tokens of autoregressive generation, each conditioned on every previous token, across 32 layers with 64 AllReduce operations per token—a total of 7,680 cross-machine reductions with zero accumulated error visible in the output. This is the theoretical floor in practice: NCCL-equivalent precision over TCP, across heterogeneous consumer GPUs, on Windows.

5 System Design

The system is a GPU-native compute mesh built on three components: an **orchestrator** for node discovery and session management, a **custom TCP transport** replacing NCCL, and a **TensorRT plugin layer** that intercepts collective operations and routes them through the transport.

Orchestrator. A central orchestrator handles node registration, session matching, and rank assignment. Daemon nodes on each machine register with the orchestrator, advertising their GPU capabilities (model, VRAM, SM count, compute

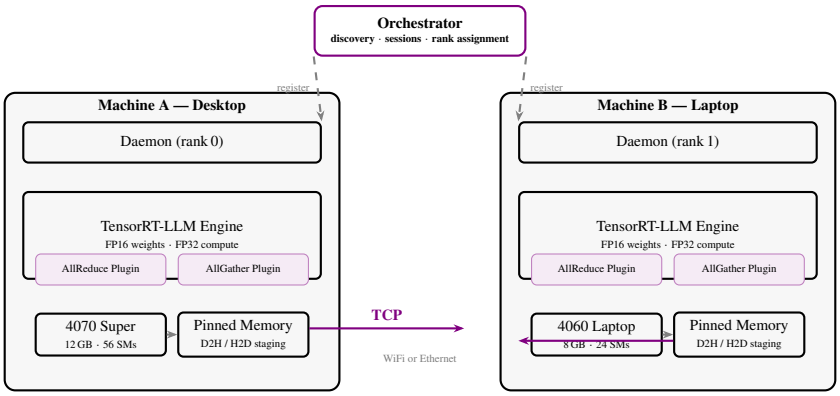

Figure 2: System architecture. The orchestrator manages topology; it does not participate in data transfer. Custom TensorRT plugins intercept AllReduce/AllGather calls and route them through the TCP transport instead of NCCL. Pinned (page-locked) host memory enables DMA-speed GPU–host transfers.

capability). When compatible nodes come online, the orchestrator creates a session and connects them. The orchestrator does not participate in data transfer—it only manages topology.

TCP transport and AllReduce. The custom transport replaces NCCL entirely. Built on WinSock2 with a POSIX socket fallback for Linux portability (untested). Each rank bootstraps by exchanging capability information, then establishing a full-mesh of TCP connections. Tensor data flows through a staged pipeline: the GPU writes output to a pinned (page-locked) host buffer via asynchronous D2H copy, the CPU sends that buffer over TCP to the peer, the peer receives into its own pinned buffer, and a CUDA kernel reduces the incoming tensor with the local partial result directly on GPU in FP32. Each rank operates independently—no barrier, no assumption of symmetric compute speed—which is what enables heterogeneous GPUs to participate in the same AllReduce without protocol-level stalls.

Pinned memory architecture. The transport pre-allocates all host buffers at initialization using `cudaHostAlloc` with `cudaHostAllocDefault`, page-locking them in physical RAM. Page-locked (pinned) memory bypasses the OS virtual memory system, enabling DMA transfers between GPU and host at the highest achievable PCIe bandwidth [NVIDIA, 2024a]—without pinning, `cudaMemcpy` must first copy to an internal pinned staging area, adding latency. The transport maintains **four pinned buffers** in a double-buffered configuration: two send/recv pairs that alternate between consecutive AllReduce calls, preventing a situation where the current call's recv overwrites the previous call's send data still in flight. A separate **GPU scratch buffer** handles the in-place reduction case (where `send_buf == recv_buf`): peer data is copied to scratch via H2D, local data remains in `recv_buf`, and a custom CUDA kernel sums them element-wise in FP32 precision. No host-side arithmetic ever touches the tensor data—all reduction is GPU-native.

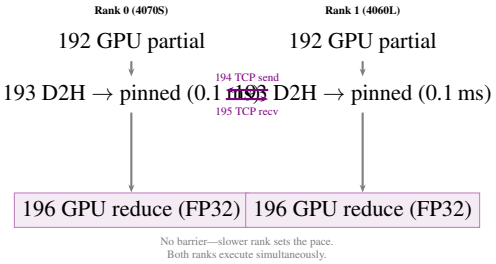

Figure 3: AllReduce data flow for TP=2 direct exchange. Both ranks simultaneously send/recv on dual TCP sockets, then reduce on GPU in FP32. For $N > 2$, the exchange becomes a ring AllReduce [Gibiansky, 2017, Patarasuk and Yuan, 2009].

Dual-socket connections. Each peer connection establishes two independent TCP sockets—one dedicated for send, one for recv. This enables truly simultaneous bidirectional transfer: while rank 0 sends its tensor to rank 1, rank 1 simultaneously sends its tensor to rank 0, on separate sockets. A single TCP socket would serialize these operations (half-duplex at the application layer even though TCP is full-duplex), doubling the exchange time. With dual sockets, the TCP exchange step completes in the time of one transfer, not two. Each socket is protected by its own mutex to prevent concurrent access from the overlapped recv thread and the main send path.

For **TP=2** (the common case), AllReduce is optimized as a direct exchange: both ranks simultaneously send their full tensor over dual TCP sockets, receive the peer's tensor into a pinned recv buffer, and sum on GPU via the custom FP32 kernel. No ring topology—just one round trip. For **larger topologies** ($N > 2$ GPUs), the system implements a ring AllReduce [Gibiansky, 2017] in two phases: reduce-scatter followed by all-gather. Total data transferred scales optimally at $2(N-1)/N \times$ tensor_size [Patarasuk and Yuan, 2009]. Send and receive are overlapped to pipeline TCP I/O with GPU computation. The ring path is implemented but untested beyond TP=2; all benchmarks in this paper use the direct exchange.

TensorRT plugin integration. TensorRT-LLM's multi-GPU support is hardcoded to NCCL—every AllReduce, AllGather, and ReduceScatter operation in the engine graph calls into NCCL's collective communication library. There is no plugin interface or abstraction layer for swapping the communication backend. To replace NCCL with our TCP transport, we implemented custom TensorRT `IPluginV2DynamicExt` plugins from scratch in C++—one for each collective operation. These plugins register under the same operation names that TRT-LLM emits during engine compilation, so they intercept every collective call at execution time without modifying TRT-LLM's model definitions, graph construction, or weight loading. The engine graph is unchanged; only the communication backend is swapped.

This required solving several integration challenges. TensorRT's plugin initialization order does not guarantee that the TCP transport is connected when the engine loads—the plugins use lazy initialization, acquiring the transport singleton on first `enqueue()` rather than at `initialize()` time. During engine compilation, TensorRT profiles each plugin with test data to select optimal execution tactics—a no-transport fallback (shifted-add) prevents TRT from dead-code-eliminating the AllReduce operations, which would silently strip them from the execution graph. The plugins handle FP16, FP32, BF16, INT8, INT32, INT64, and FP8 data types, matching TRT-LLM's full dtype coverage for collective operations.

6 TensorRT-LLM on Windows

NVIDIA discontinued Windows support for TensorRT-LLM. We restored it on v0.12.0, the last release with partial Windows compatibility, and the latest at time of development. This is a prerequisite for consumer deployment: the vast majority of the 100M+ RTX GPUs cited in §1 run Windows, and requiring Linux would exclude most of the target hardware.

The port required patches at four layers of the build system: **Package management**—Conan dependency profiles assumed Linux toolchains; we wrote Windows-native profiles for all C++ dependencies. **FMHA kernel compilation**—fused multi-head attention kernels used GCC-specific intrinsics and POSIX paths; we ported these to MSVC with equivalent CUDA intrinsics. **Python bindings**—nanobind's build configuration assumed Unix shared library conventions; we patched the CMake targets for Windows DLL generation. **MSVC/CUDA interop**—the CUDA toolkit and MSVC have different C++ ABI expectations for exception handling and name mangling; we resolved linking failures across the toolchain boundary.

The result is a native Windows build that compiles and executes TensorRT-LLM inference engines—the full set of NVIDIA's 1,500+ hand-tuned CUDA kernels for attention, GEMM, and quantization—without WSL, Docker, or any Linux compatibility layer.

7 Experimental Evaluation

We demonstrated tensor parallelism across two heterogeneous GPUs on separate machines:

- **Rank 0:** NVIDIA RTX 4070 Super—12 GB VRAM, 56 SMs
- **Rank 1:** NVIDIA RTX 4060 Laptop—8 GB VRAM, 24 SMs

These GPUs differ in VRAM, SM count, and clock speed—a configuration NCCL cannot support. Both GPUs share the same SM architecture (Ada Lovelace, sm_89)—heterogeneous TP has only been confirmed across GPUs with identical SM architectures. Cross-architecture TP (*e.g.*, Ampere + Ada) within the same TP group remains untested and may introduce divergence from differing instruction-level behavior. In MoE configurations, different experts can run on different architectures freely since experts never AllReduce with each other—the constraint applies only within each expert's TP group.

Experimental setup. Software: NVIDIA TensorRT-LLM v0.12.0 (custom Windows build), CUDA Toolkit 12.6, cuDNN 9.3, TensorRT 10.4, Python 3.12, Windows 11 Pro on both machines. Engines built with `trtllm-build` using TP=2, FP16 weights with FP32 AllReduce accumulation. **Network:** WiFi tests used 802.11ac (5 GHz), measured ping 316 ms round-trip under load. Ethernet tests used gigabit (1000BASE-T) via consumer router, measured ping 0.5–1 ms. **Methodology:** All latency measurements are per-token generation time during autoregressive decoding, measured after a 5-token warm-up period to ensure CUDA kernels are JIT-compiled and GPU clocks are boosted. Each reported number is the median of 20 consecutive tokens. Throughput is computed as $1/\text{latency}$. AllReduce timing breakdowns use CUDA event-based instrumentation inserted at each stage of the D2H $\rightarrow$ TCP $\rightarrow$ H2D $\rightarrow$ reduce pipeline.

7.1 Correctness—TinyLlama 1.1B

TinyLlama 1.1B (22 layers, FP32) was split horizontally, with each rank holding half the weights. Every transformer layer synchronizes via our custom TCP AllReduce with FP32 accumulation. Verified outputs across both ranks:

- **"1+1="** $\rightarrow$ "2" — both ranks agree ✓
- **"The capital of France is"** $\rightarrow$ "Paris" — both ranks agree ✓
- **"A cat is a"** $\rightarrow$ "domestic" — both ranks agree ✓

Identical outputs from two different GPUs, on two different machines. FP32 GPU-native reduction is what makes this possible—without it, accumulated precision loss corrupts the output by layer 15.

7.2 Scale—Mistral 7B

To prove the architecture scales beyond toy models, we built and deployed **Mistral 7B Instruct**—a 7 billion parameter model with 32 transformer layers, grouped-query attention (8 KV heads), and 14 GB of FP16 weights. This model **cannot fit on either GPU alone**: the 4070 Super has 12 GB, the 4060 Laptop has 8 GB. Without tensor parallelism, this model simply doesn't run on consumer hardware with less than 16 GB VRAM.

Table 3: Performance comparison across configurations.

Configuration	Latency	Tput	Notes
llama.cpp — 4070S single	3.4 ms	295 t/s	Q8, single GPU baseline
BMRT TP=2 — localhost	8.1 ms	123 t/s	TinyLlama, no network
BMRT TP=2 — WiFi	277 ms	3.6 t/s	TinyLlama, 316 ms ping
BMRT TP=2 — Ethernet	276 ms	3.6 t/s	TinyLlama, 1 ms ping
BMRT TP=2 — Mistral (early)	185 ms	5.4 t/s	Overlap, 49 KB payload
BMRT TP=2 — Mistral	**80 ms**	**12.5 t/s**	**KV cache, 8 KB payload**

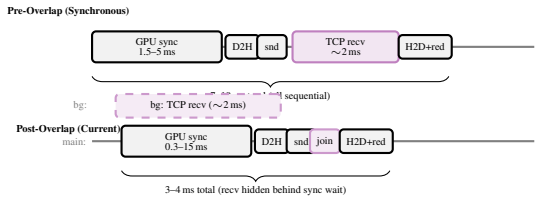

Figure 4: AllReduce timeline before and after overlap optimization. Pre-overlap: all stages sequential (7–10 ms). Post-overlap: TCP recv runs on a background thread during GPU sync wait; when sync_wait > recv_time, the recv adds zero time to the critical path.

With TP=2 across both machines, Mistral 7B produces coherent output:

> **"What is the capital of France?"** → "The capital of France is Paris."
> Generation latency: **80 ms/token (12.5 tok/s)** over gigabit ethernet with KV cache + overlapped AllReduce.

7.3 Benchmark Data

7.4 Analysis: Network Latency vs. Synchronization Overhead

Before overlap optimization: gigabit ethernet (1 ms ping) and WiFi (316 ms ping) produced identical throughput—276 ms vs. 277 ms per token. A 300× improvement in raw network latency yielded zero improvement in token generation speed. This initially appeared to contradict the assumption that network latency makes per-layer synchronization infeasible.

Instrumentation revealed why. In the synchronous (pre-overlap) implementation, each AllReduce call spent 1.5–5 ms waiting for GPU compute to finish before any network transfer began. This synchronization wait dominated the per-call cost, masking the actual network transfer time. WiFi and ethernet produced identical results because neither was the bottleneck—the GPU sync wait was.

After overlap optimization: we start the TCP recv in a background thread *before* waiting for GPU compute, so recv runs in parallel with the sync wait. With double-buffered pinned memory to prevent send/recv conflicts, the overlapped AllReduce hides recv time behind the sync wait on the critical path.

When the GPU sync wait (0.3–15 ms) exceeds the recv time (∼2 ms), the recv completes before the main thread needs it—the join is instant and recv adds zero time to the critical path. The effective per-call cost becomes: sync_wait + D2H (0.1 ms) + send (0.1 ms) + join (0 ms) + H2D+reduce (0.3 ms) ≈ sync_wait + 0.5 ms. At 3–4 ms per call × 64 calls per token (Mistral 7B, 32 layers with AllReduce + AllGather), this yields ∼185 ms/tok.

This reduced per-call AllReduce from 7–10 ms to 3–4 ms and improved Mistral 7B generation from 293 ms to **185 ms/tok (5.4 tok/s)**—a 37% latency reduction. Full layer-level pipelining (deferring recv+reduce to the next AllReduce call) was attempted but breaks TRT-LLM's buffer contract—the engine expects the reduced result in `recv_buf` before `allReduce()` returns.

KV cache effect on AllReduce payload. With correct KV caching, the generation phase processes a single token per step instead of re-computing the full context. This shrinks the AllReduce payload from ∼49 KB (full activation tensor) to **8 KB** (single-token hidden state). At 8 KB, each AllReduce call completes in ∼1.2 ms—down from 3–4 ms at 49 KB. Over 64 calls per token, this reduces AllReduce overhead from ∼224 ms to ∼77 ms. Combined with faster single-token GPU compute, generation latency drops to **80 ms/tok (12.5 tok/s)**—a 2.3× improvement over the pre-KV-cache measurement.

Optimization trajectory. The three benchmark stages tell a story about where the remaining headroom lies:

1. **293 ms/tok** — Synchronous AllReduce. Recv blocks the critical path. GPU sync wait (1.5–5 ms) + network transfer (∼2 ms) = 7–10 ms per call × 64 calls.
2. **185 ms/tok** — Overlapped AllReduce. Recv runs during GPU sync wait. Per-call cost drops to 3–4 ms. A 37% latency reduction from hiding I/O behind compute.
3. **80 ms/tok** — KV cache fix. Generation-phase payload shrinks from 49 KB to 8 KB. Per-call cost drops to ∼1.2 ms. A further 57% reduction from sending less data.

Each stage removed a different bottleneck: first the blocking recv, then the payload size. The remaining bottleneck is the GPU sync wait itself (∼0.7 ms per call), which is the time for each layer's TensorRT compute to finish—the irreducible cost of the model's arithmetic. Further gains require either reducing per-layer compute (quantization, pruning) or overlapping the next layer's compute with the current layer's reduction (layer-level pipelining, which we attempted but is blocked by TRT-LLM's buffer contract). Larger models improve the ratio further: more compute per layer means more sync wait to hide communication behind, making AllReduce overhead a smaller fraction of per-token cost.

To contextualize: 12.5 tokens/second is not competitive with single-GPU inference on a model that fits in VRAM—`llama.cpp` on one 4070 Super runs a quantized model at 295 tok/s. But TP and single-GPU address fundamentally different constraints. Single-GPU runs models that fit on one card. TP splits a model that **cannot fit on any single GPU** across several for capability—and at 12.5 tok/s, the output streams faster than a human reads. The throughput is practical for interactive use, and the correctness result—identical output from mismatched GPUs over a commodity network—is the primary contribution.

8 Limitations

Correctness is proven across two model scales. Overlapped AllReduce (§7.4) improved Mistral 7B latency from 293 ms to 80 ms/tok. The remaining optimizations:

Further computation-communication overlap — the current implementation overlaps TCP recv with GPU sync wait by starting recv in a background thread. Full layer-level pipelining—deferring the previous layer's recv+reduce while the next layer's compute begins—was attempted but breaks TRT-LLM's buffer contract. Achieving this would require modifying TensorRT's execution model or introducing a shadow buffer scheme.

Larger models improve the ratio — Mistral 7B (32 layers, 14 GB) achieves nearly the same per-token latency as TinyLlama (22 layers, 2 GB) despite $10\times$ more parameters. Each layer has more compute relative to the fixed AllReduce overhead. At 70B+ parameters, the AllReduce cost becomes a rounding error—exactly where TP shines.

Same SM architecture required within a TP group — heterogeneous TP has been confirmed only across GPUs sharing the same SM architecture (Ada Lovelace, sm_89). Cross-architecture pairs (*e.g.*, Ampere sm_86 + Ada sm_89) within the same TP group remain untested. In MoE configurations, this constraint applies only within each expert's TP group—different experts can run on different architectures since they never AllReduce with each other.

Asymmetric weight splitting — currently, TP splits weight matrices evenly. With heterogeneous GPUs, proportional splitting (more columns to the GPU with more VRAM) would allow larger models to fit across mismatched hardware. The transport and reduction layers are already agnostic to split ratios.

Concurrent serving — dense TP locks all ranks for every token. The current system serves one user at a time. Continuous batching—processing multiple users' tokens in the same forward pass—would amortize AllReduce overhead across N users while growing the payload only linearly ($8\,\text{KB} \times N$). TRT-LLM supports this on Linux via its C++ executor; enabling it over our TCP transport requires adapting the scheduling logic. MoE architectures (§9) solve this structurally: only 2 of 8 experts activate per token, so 75% of the mesh can serve other users concurrently without batching.

We prioritized proving correctness over the most challenging transport—WiFi between consumer laptops. The benchmarks demonstrate that the transport is already saturated: faster networks yield no throughput improvement because the network is not the bottleneck. **The remaining gains lie in overlapping compute with communication and in model architectures that reduce synchronization points—both engineering optimizations rather than open research problems.**

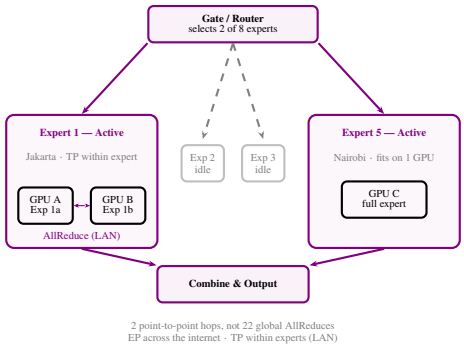

Figure 5: MoE hybrid parallelism. The gate routes each token to 2 of 8 experts. Expert parallelism (EP) uses point-to-point TCP across the internet; tensor parallelism (TP) uses AllReduce within local GPU clusters. 75% of experts are idle per token—available for concurrent serving.

9 Future Work

9.1 Mixture-of-Experts and Edge Compute

Dense transformer models are the worst case for edge tensor parallelism—every layer requires AllReduce across all ranks, every token. But the industry is moving to **Mixture-of-Experts (MoE)** architectures, and this changes the math entirely.

In a MoE model like Mixtral [Jiang et al., 2024a], each transformer layer contains multiple expert sub-networks, but only 2 of 8 experts activate per token. A gating network routes each token to its best-matching experts. The rest stay idle. This means most of the model's parameters exist but don't participate in any given forward pass—a property that is inefficient in a data center but **advantageous for a distributed compute mesh**.

Instead of splitting one dense model across GPUs that must synchronize every layer, each expert lives on a different node. A token arrives, the gate selects two experts, and only those two nodes compute. No AllReduce across all ranks—just point-to-point communication between the gate and the selected experts. The per-layer synchronization bottleneck that makes dense TP painful over WiFi largely disappears.

The compute mesh topology becomes an advantage, not a liability. Experts can be placed geographically—routed to the nearest available GPU with that expert hot in VRAM, like a CDN for inference. A gamer in Jakarta holds Expert 3, a researcher in Nairobi holds Expert 7. A token that needs both pays two point-to-point hops, not a global AllReduce.

The two parallelism strategies also compose. If a single expert is too large for one GPU, it can be tensor-parallelized across nearby GPUs—TP within the expert, expert parallelism across the mesh. This creates two tiers of communication: **EP across the internet** for routing tokens to the right expert (point-to-point, latency-tolerant, one hop), and **TP across local GPUs** for computing within the expert (AllReduce over ethernet at 0.1 ms, not WiFi at 5 ms). Since only 2 of 8 experts activate per token, 75% of the mesh is idle for any given request—available to serve other tokens concurrently.

Data center architecture is overprovisioned for MoE. Dense transformers activate every parameter on every token—they fully utilize NVLink's 900 GB/s bandwidth and justify the cost of keeping every GPU in a DGX node busy. MoE inverts this: 75% of parameters are idle per token, and the communication pattern is sparse point-to-point, not dense AllReduce. Data centers compensate by batching aggressively, but this requires sustained high request volume. A $3M GB200 NVL72 rack has 75% of its 13.8 TB VRAM idle per token. A mesh inverts the cost structure: idle VRAM costs nearly nothing because it sits in consumer hardware that would otherwise be sleeping, and expert routing needs only commodity TCP. Moreover, data centers scale beyond NVLink via pipeline parallelism—a degraded mechanism that increases sequential depth and introduces bubbles. MoE on a mesh scales via expert parallelism: each expert occupies its own node(s), aggregate VRAM grows linearly with mesh size (100 nodes $\times$ 12 GB $=$ 1.2 TB), and per-token communication cost does not increase. **Data centers scale capacity by adding pipeline stages. The mesh scales capacity by adding participants. One degrades utilization; the other does not.**

Mesh formation. The orchestrator already solves the assignment problem for dense TP: nodes register, advertise GPU capabilities, and receive rank assignments. MoE extends this with expert placement. When a node joins the mesh, the orchestrator evaluates its VRAM capacity, network latency to existing nodes, and which experts are underserved—then assigns one or more experts and pushes the corresponding weight shards. If an expert is too large for a single GPU, the orchestrator groups nearby low-latency nodes into a TP group for that expert, exactly as it does today for dense models.

Redundancy and fault tolerance. Dense TP has no tolerance for node failure—if any rank drops, the entire inference pass is invalid. MoE on a mesh offers a structural alternative. Since only 2 of 8 experts activate per token and 75% of nodes are idle at any moment, the orchestrator can maintain **replicated experts**—the same expert weights loaded on multiple nodes in different locations. When the orchestrator's heartbeat detects a node failure, it re-routes subsequent tokens to a surviving replica. No checkpoint, no restart, no lost inference state—the next token simply goes elsewhere.

Concurrent serving. Dense TP locks every GPU into a single inference pass. MoE on a mesh does not: since only 2 of 8 experts activate per token, 75% of nodes can simultaneously serve other users' tokens. Multiple inference requests flow through the mesh concurrently without explicit batching or model replication—the sparsity that makes MoE parameter-efficient also makes it serving-efficient.

Our transport layer already supports arbitrary world sizes and rank-to-rank communication. The infrastructure for MoE routing is the same TCP mesh we built for tensor parallelism—the difference is traffic pattern, not protocol. **Dense TP validates the transport layer; MoE architectures are where it achieves practical scalability.**

9.2 Distributed KV Cache

Foundation models increasingly support context windows of 128K–1M+ tokens. The KV cache required to serve these contexts grows linearly with sequence length—at 128K tokens, Mistral 7B's KV cache alone consumes 4–8 GB per GPU, approaching or exceeding the total VRAM of consumer hardware. This is the next wall after model size: even when tensor parallelism enables a model's weights to fit across multiple GPUs, a long context window can exhaust the remaining VRAM on the smallest device.

Existing solutions trade latency for capacity. FlexGen [Sheng et al., 2023] offloads KV cache to CPU RAM and disk, solving the memory problem but adding 10–50 ms per page fault. Ring Attention [Liu et al., 2023] distributes the sequence across devices, requiring all devices to participate in every attention computation—a global synchronization cost that scales with world size. PagedAttention (vLLM) manages KV cache as virtual memory pages on a single GPU but cannot help when that GPU is full.

Our mesh architecture enables a different approach: **distributed KV cache across idle nodes**. The orchestrator already tracks every node's GPU model, VRAM capacity, and current utilization. The TCP transport already moves tensors between GPUs with pinned-memory staging. In an MoE configuration, 75% of expert nodes are idle for any given token—their VRAM is empty and available. Instead of evicting old KV entries to CPU or dropping context, we propose paging them to peer GPUs in the mesh that have free VRAM.

This creates a three-tier memory hierarchy for KV cache:

- **Tier 1—Local GPU VRAM** (hot): Recent tokens, zero additional latency.
- **Tier 2—Peer GPU VRAM** (warm): Older tokens paged to idle nodes. Retrieval cost is one TCP round trip (1–5 ms over ethernet)—slower than local VRAM but faster than disk I/O (10–50 ms).
- **Tier 3—CPU RAM / disk** (cold): Oldest tokens, last resort.

The critical difference from existing offloading schemes is that Tier 2 uses **GPU VRAM on peer nodes**, not local CPU RAM. A fetch from peer GPU VRAM over ethernet is a DMA-speed copy on both ends with a network hop in between—the same staged pipeline (GPU $\rightarrow$ pinned host $\rightarrow$ TCP $\rightarrow$ pinned host $\rightarrow$ GPU) already proven in our AllReduce implementation. Unlike Ring Attention, only the node that needs a specific KV block fetches it—no global synchronization, just point-to-point retrieval.

In MoE configurations, this composes naturally with expert routing. The gate routes a token to two active experts; the remaining six expert nodes are idle, their combined VRAM forming a distributed memory pool. The same orchestrator that routes computation can route KV cache placement—assigning pages to the nearest node with free VRAM, like a CDN for memory.

Three prerequisites make this approach viable: (1) an orchestrator with real-time visibility into per-node VRAM utilization, (2) a transport layer capable of GPU-to-GPU tensor transfer across the network, and (3) idle nodes with empty VRAM available as storage. Our system already satisfies all three. The remaining work is a cache eviction policy (LRU

with locality-aware placement) and integration with TensorRT-LLM's attention kernels to support non-contiguous KV cache reads. **The same mesh that routes computation can also route memory, reducing dependence on centralized cloud infrastructure.**

10 Conclusion

We presented the first implementation of heterogeneous tensor parallelism across consumer GPUs on separate machines over commodity TCP networks. By solving two problems that blocked prior attempts—precision loss during networked AllReduce and synchronization failure across mismatched GPUs—we demonstrated that tensor parallelism is feasible outside of homogeneous data center hardware. Our benchmarks revealed that in synchronous tensor parallelism, GPU synchronization overhead—not network latency—is the dominant bottleneck: a $300\times$ improvement in network latency yielded zero improvement in throughput. Overlapping TCP recv with GPU synchronization reduced per-call AllReduce cost by 50–60%. Combined with correct KV caching—which shrinks the generation-phase AllReduce payload from 49 KB to 8 KB—Mistral 7B generation improved from 293 ms to **80 ms/tok (12.5 tok/s)**.

The system produces identical output from two different GPUs (RTX 4070 Super and RTX 4060 Laptop) running Mistral 7B—a 14 GB model too large for either GPU alone—over gigabit ethernet. The industry's shift toward Mixture-of-Experts architectures, where only a fraction of parameters activate per token, structurally favors distributed meshes over centralized data centers: expert routing requires only point-to-point communication over commodity networks, not AllReduce over NVLink, and data centers scale past their NVLink domains via pipeline parallelism—a degraded mechanism that reintroduces the bubbles and idle GPUs that tensor parallelism was designed to eliminate. A mesh scales via expert parallelism, which does not degrade. Dense tensor parallelism proved the transport works. Mixture-of-Experts is where it becomes practical.

References

Alexander Borzunov et al. Petals: Collaborative inference and fine-tuning of large models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. System Demonstrations.

Exo Explore. exo: Run your own AI cluster at home with everyday devices. https://github.com/exo-explore/exo, 2024.

Andrew Gibiansky. Bringing HPC techniques to deep learning. Baidu Research, 2017.

Yanping Huang et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Albert Q. Jiang et al. Mixtral of experts. *arXiv:2401.04088*, 2024a.

Youhe Jiang et al. HexGen: Generative inference of foundation model over heterogeneous decentralized environment. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024b.

Sungmin Lee et al. AMP: Automatically finding model parallel strategies with heterogeneity awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Hao Liu et al. Ring attention with blockwise transformers for near-infinite context. *arXiv:2310.01889*, 2023.

Xin Mei et al. Helix: Distributed serving of large language models via max-flow on heterogeneous GPUs. In *Proceedings of the 30th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

NVIDIA. CUDA C++ best practices guide: Data transfer between host and device. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide, 2024a.

NVIDIA. NCCL: NVIDIA collective communications library. https://developer.nvidia.com/nccl, 2024b.

NVIDIA. Over 100 million RTX AI PCs and workstations. NVIDIA Newsroom, CES 2024, 2024c.

NVIDIA. TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM, 2024d.

Pitch Patarasuk and Xin Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2), 2009.

Jeff Rasley et al. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020.

Ying Sheng et al. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

Mohammad Shoeybi et al. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv:1909.08053*, 2019.

Lianmin Zheng et al. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.